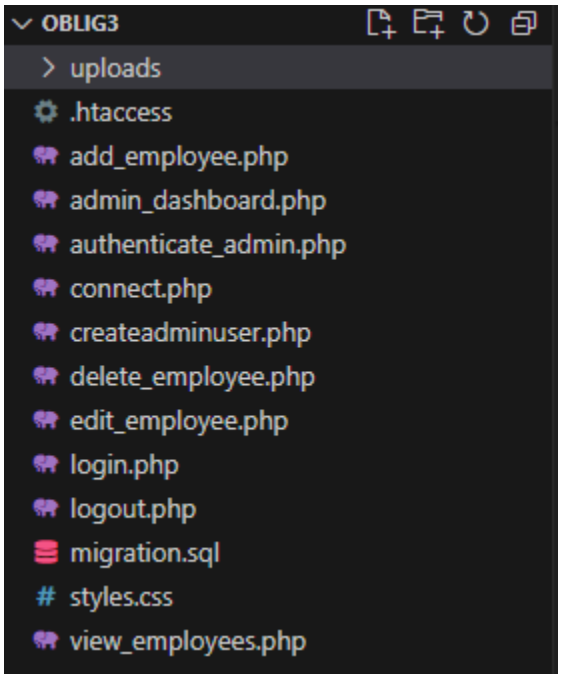
Project Structure:

The application is structured in a modular way, meaning each operation is handled by a separate file. For example, one file handles adding the employees, while other operations such as editing, viewing, or deleting employees are separated in different files.



The file names are meant to be easily understandable and describe what each file does, or is.

Functionality like establishing a connection to the database is handled by connect.php, and authenticating that the user is signed in as an admin is handled by authenticate_admin.php. This is done by having the admin sign in through the login page, checking their credentials against the username and hashed password stored in the database, and then maintaining their session until they sign out. If a page is accessed without a valid admin session, they will be redirected to the login page.

I've disabled direct access to these files (connect and authenticate) by adding a .htaccess file so Apache will block browsers from navigating to them, this is just for some added security. They are separated to follow modular design and DRY, so there is no reason these should be opened in the browser. They are purely for the other scripts to use.

Assumptions:

While the slides from class involve checking file size and file type for when uploading photos, I felt this was not directly necessary in this case. While setting file size is useful for pages where a normal user is uploading the file, for a page where the admin is doing it this might be more annoying than helpful. An admin should be fully aware of what they're uploading unlike a normal user, and the admin might want to upload an image that is larger than what I set, for example 5MB, or 10MB. If I implement the file size check, I'm just restricting their ability to use their application, without providing much value, at least in this scenario.

The same thing applies for the file type, the upload button already asks the admin for an image file. Unless my intention is to restrict this for security reasons I don't see why I would reduce the admin's autonomy to select the file type they want. For example, if I set it to JPG, JPEG or PNG, and the admin later wants to use a WEBP file, which is also an image, they won't be able to, even though there is no good reason why they shouldn't. That means the application would need to be updated when the admin meets a restriction I didn't think of when making it, rather than giving them the autonomy to make an informed decision on their own. I have instead focused on ensuring that the filenames are unique and that they don't contain special characters to prevent any potential conflicts with the database. I also ensured that images in /uploads are deleted when a new photo is uploaded through edit employee, or the employee is deleted.

Database Schema:

The database schema is included in the migration.sql file, which is located in the Oblig3 directory. However, I'll include the text version here.

```sql
-- creating the database if it doesn't exist
CREATE DATABASE IF NOT EXISTS CORAX_db;

-- selecting corax_db
USE CORAX_db;

-- creating admin table
CREATE table admin (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL
);

-- creating employees table
CREATE table employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT NOT NULL,
    job_title VARCHAR(100) NOT NULL,
    department VARCHAR(100) NOT NULL,
    photo_path VARCHAR(255) DEFAULT NULL
);
```

How to run the application:

To run the application you will need a functional MySQL and Apache installation, for example through XAMPP.

First, move the application (oblig3 folder) to the htdocs directory of your XAMPP installation so Apache can use it, then create the database by going to your MySQL PHPMyAdmin and importing the “migration.sql” file found inside the Oblig3 folder. This will initialize the required tables and database.

Proceed to create the admin user. This can be done by running the “createadminuser.php” file also found in the Oblig3 directory. To do this you can just load the script in the browser by navigating to it, or execute it in VS Code, etc. If you wish you can edit the username and password, but once this file has been used to create the admin user, delete it, or remove it from the file structure. It should not be available in the directory after being set up.

You now have the admin user and the database, and can proceed to log in. Enter the username and password you chose for the admin user in the previous file, or the one that is already configured in the file. Once you’re logged in, you can add employees, view the current list of employees, or log out of the application. Always remember to log out when you’re done, to ensure your session ends when you exit the application.

To add a new employee, simply click “Add new employee” from the admin dashboard and fill out the fields. All fields are required, and an ID for the employee will be automatically assigned by the database.

After adding an employee you will be redirected to the employee list. If you wish to add another employee, click add new employee at the top of the page. If you wish to go back to the dashboard, you can click that link instead.

If you wish to edit or delete the employee, you can do so at the right hand of the page. If you press Edit, you will retrieve the current details of the employee and have the option to edit them. Note that you don’t need to upload a new image. If you don't, the employee will keep their current image. If you upload a new one, the old one will be deleted and the employee will use the new one instead.

If you wish to delete an employee, simply press the delete button and they will be removed from the database, and their image will be deleted from the uploads folder.